Ascent
FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

Cours apprenant

ReST API — bonnes pratiques et sécurité

MODULE 3

Menaces et vulnérabilités des API REST

OWASP API Top 10, modèles de menace, audit avec OWASP ZAP et Burp Suite

CRÉATEUR DU COURS
Boris Rose
Formateur & concepteur du contenu

Table des matières

Avant-propos **2**

1 Modéliser les menaces avant de coder **2**
1.1 Les quatre questions d'Adam Shostack . . . 2
1.2 Le modèle STRIDE . . . 3
1.3 Frameworks complémentaires de threat modeling . . . 3
1.3.1 PASTA (Process for Attack Simulation and Threat Analysis) . . . 3
1.3.2 LINDDUN . . . 4
1.3.3 MITRE ATT&CK . . . 5
1.3.4 Frameworks moins centraux mais utiles à connaître . . . 5

2 Le top 10 des risques OWASP API (2023) **6**
2.1 API1 – BOLA (Broken Object Level Authorization) . . . 6
2.2 API3 – BOPLA (Mass assignment et Excessive data exposure) . . . 6
2.3 API7 – SSRF (Server-Side Request Forgery) . . . 7
2.4 API4 – Unrestricted Resource Consumption . . . 7

3 Attaques côté navigateur **7**
3.1 XSS (Cross-Site Scripting) . . . 7
3.2 CSRF (Cross-Site Request Forgery) . . . 8
3.3 CORS mal configuré . . . 8

4 Mauvaises configurations classiques **8**

5 CVSS : noter une vulnérabilité **8**

6 Les outils d'audit **9**
6.1 OWASP ZAP . . . 9
6.2 Burp Suite . . . 9
6.3 Burp Active Scan — au sein de Burp Suite . . . 10
6.4 Outils de scan complémentaires . . . 10
6.5 Plateformes vulnérables pour s'entraîner légalement . . . 13

7 Démarche d'audit en équipe **14**

8 Aspects légaux à connaître **14**
8.1 Plateformes de bug bounty . . . 15

9 Synthèse **15**

10 Lexique du module **16**

11 Pour aller plus loin **17**

Avant-propos

Ce cours est le compagnon écrit du **Module 3**. Il développe les attaques classiques sur les API REST, présente les frameworks de *threat modeling* et décrit les outils d'audit que vous utiliserez en TP. *Cours conçu et rédigé par Boris Rose, formateur.*

1 Modéliser les menaces avant de coder

Définition

Threat modeling : démarche structurée qui consiste à identifier en amont les menaces qui pèsent sur un système, leurs conséquences potentielles et les mesures de protection à prévoir. À pratiquer dès la conception, pas après le déploiement.

1.1 Les quatre questions d'Adam Shostack

Définition

Adam Shostack : ingénieur sécurité américain, ancien architecte sécurité chez Microsoft (2006–2014) où il a popularisé STRIDE et la pratique du *threat modeling* dans le cycle de développement. Auteur du livre de référence *Threat Modeling: Designing for Security* (Wiley, 2014), aujourd'hui consultant indépendant et enseignant à l'université de Washington. Ses « quatre questions » sont devenues le canevas standard pour structurer un atelier de threat modeling.

1. **Qu'est-ce qu'on construit ?** (diagramme de flux de données, frontières de confiance)
2. **Qu'est-ce qui peut mal tourner ?** (modèle STRIDE)
3. **Qu'est-ce qu'on fait à ce sujet ?** (mitiger, transférer, accepter, éliminer)
4. **A-t-on bien fait ?** (revues, tests d'intrusion, scans automatisés)

Exemple

Atelier d'1 heure sur une nouvelle API « commande ».

1. *Qu'est-ce qu'on construit ?* L'équipe dessine au tableau : navigateur → API → base de commandes → service paiement → service mail. Frontières de confiance entre chaque composant.
2. *Qu'est-ce qui peut mal tourner ?* Pour chaque flèche, on passe en revue les six lettres STRIDE. On note 18 menaces.
3. *Qu'est-ce qu'on fait ?* 12 menaces sont mitigées (TLS, JWT, validation), 4 transférées (au PSP), 2 acceptées (risque résiduel faible).
4. *A-t-on bien fait ?* Inscription d'une revue trimestrielle au backlog, scan ZAP en CI, pen-test avant le go-live.

1.2 Le modèle STRIDE

Définition

STRIDE (Microsoft, années 1990) : taxonomie de menaces en six catégories. C'est le framework le plus simple et le plus utilisé.

Lettre	Menace	Définition
S	Spoofing	Usurper l'identité d'un utilisateur, d'un service.
T	Tampering	Altérer des données en transit ou au repos.
R	Repudiation	Pouvoir nier une action faute de traces fiables.
I	Info disclosure	Exposer une information à qui ne devrait pas la voir.
D	DoS	Indisponibilité partielle ou totale du service.
E	Elevation of priv.	Acquérir des droits supérieurs aux droits légitimes.

1.3 Frameworks complémentaires de threat modeling

Au-delà de STRIDE, plusieurs autres frameworks structurés existent pour modéliser les menaces. Chacun a sa philosophie et son public privilégié. Voici les trois les plus couramment cités, avec pour chacun une définition claire, ce qu'il apporte, comment il fonctionne et un exemple concret.

1.3.1 PASTA (Process for Attack Simulation and Threat Analysis)

Définition

PASTA : framework de threat modeling créé en 2012 par Tony UcedaVélez et Marco Morana, plus complet que STRIDE et orienté *risque métier*. Il déroule sept étapes successives qui partent des objectifs de l'entreprise et descendent jusqu'au code, en simulant le point de vue d'un attaquant à chaque étape.

Les 7 étapes de PASTA :

1. Définir les objectifs (business objectives, exigences réglementaires).
2. Définir le périmètre technique (composants, technologies, frontières).
3. Décomposer l'application (flux de données, dépendances).
4. Analyser les menaces (sources d'attaque, motivations).
5. Analyser les vulnérabilités (faiblesses connues, défauts de conception).
6. Modéliser les attaques (chemins exploitables, scénarios).
7. Analyser le risque et l'impact métier (chiffrer la perte potentielle).

Quand l'utiliser ? Quand vous travaillez sur une application stratégique pour l'entreprise (paiement, santé, données personnelles à fort enjeu) et que vous devez faire dialoguer équipes techniques, sécurité et direction métier. PASTA force le rapprochement.

Exemple

Application : API de virement bancaire.

1. **Objectifs** : permettre les virements 24/7 ; conformité DSP2 ; pertes financières acceptables $< 0{,}01\,\%$.
2. **Périmètre** : API REST, base PostgreSQL, IdP Keycloak, partenaire SWIFT.
3. **Décomposition** : flux d'authentification, flux d'initiation, flux de validation 3D-Secure,

flux d'envoi vers SWIFT.

4. **Menaces** : fraude par usurpation d'identité, attaque interne, fraude au président.
5. **Vulnérabilités** : MFA contournable, journaux d'audit incomplets, jetons de session longue durée.
6. **Attaques** : compromission de session + virement vers compte mule en moins de 5 minutes.
7. **Impact** : perte directe (montant viré) + amende régulateur + atteinte à l'image.

1.3.2 LINDDUN

Définition

LINDDUN : framework de threat modeling spécialisé dans la *vie privée*, créé en 2010 par l'université de Louvain (KU Leuven). Là où STRIDE couvre la sécurité, LINDDUN couvre les risques de violation des données personnelles. Son nom est l'acronyme de ses sept catégories.

Les 7 menaces LINDDUN :

- **L** — *Linkability* : possibilité d'établir un lien entre deux actions ou données du même utilisateur sans qu'il l'ait autorisé.
- **I** — *Identifiability* : possibilité d'identifier un utilisateur à partir de données pourtant censées être anonymes.
- **N** — *Non-repudiation* (au sens vie privée) : impossibilité pour l'utilisateur de nier avoir fait une action (l'inverse du besoin sécurité).
- **D** — *Detectability* : capacité d'un attaquant à détecter qu'une donnée existe (par exemple : « cet utilisateur est-il client ? »).
- **D** — *Disclosure of information* : exposition de données personnelles à des tiers non autorisés.
- **U** — *Unawareness* : utilisateur qui ignore quelles données sont collectées sur lui.
- **N** — *Non-compliance* : violation des règlements en vigueur (RGPD en Europe, CCPA en Californie, HIPAA pour la santé).

Quand l'utiliser ? Toute application qui collecte ou traite des données personnelles — en particulier celles soumises au RGPD. LINDDUN est aujourd'hui la référence pour les analyses d'impact (*DPIA*) demandées par la CNIL.

Exemple

Application : API d'une plateforme de covoiturage.

- **L** (Linkability) : si l'API renvoie systématiquement le même identifiant utilisateur à chaque trajet publié, un observateur peut reconstituer toute l'historique de mobilité d'une personne.
- **I** (Identifiability) : la combinaison « ville de départ + heure + véhicule » suffit à ré-identifier un conducteur même si son nom est anonymisé.
- **D** (Disclosure) : un endpoint `GET /users/{id}/trips` qui ne vérifie pas l'autorisation expose tout l'historique d'une victime.
- **U** (Unawareness) : l'utilisateur ne sait pas que ses coordonnées GPS sont conservées pendant 3 ans pour entraîner un modèle de pricing.
- **N** (Non-compliance) : pas de mécanisme pour exporter ou supprimer ses données = violation RGPD article 17 (« droit à l'oubli »).

1.3.3 MITRE ATT&CK

Définition

MITRE ATT&CK (*Adversarial Tactics, Techniques and Common Knowledge*) : base de connaissance maintenue par l'organisation à but non lucratif américaine MITRE depuis 2013. Elle catalogue les *tactiques* (objectifs poursuivis par un attaquant) et les *techniques* (façons concrètes de les atteindre) observées dans des incidents réels. Aujourd'hui plus de 200 techniques documentées, chacune avec exemples, mitigations et détections associées.

Structure d'ATT&CK :

- **Tactiques** (le *pourquoi*) : 14 catégories qui suivent le cycle de vie d'une attaque, par exemple *Initial Access*, *Execution*, *Persistence*, *Privilege Escalation*, *Defense Evasion*, *Credential Access*, *Discovery*, *Lateral Movement*, *Collection*, *Exfiltration*, *Impact*.
- **Techniques** (le *comment*) : actions précises, identifiées par un code (exemple : `T1190 - Exploit Public-Facing Application`).
- **Sous-techniques** : variantes plus fines (`T1190.001 - Web Application Vulnerability`).
- **Acteurs (Groups)** et **logiciels malveillants (Software)** : profils des groupes d'attaque connus et de leurs outils, avec les techniques qu'ils privilégient.

Quand l'utiliser ? Pour passer du « risque théorique » à des scénarios réels documentés. ATT&CK est devenue la référence des équipes *blue team* (défense) pour mapper leurs détections, et des *red teams* pour planifier leurs exercices. Bibliothèque consultable en ligne sur `https://attack.mitre.org`.

Exemple

Mapping ATT&CK pour une attaque sur une API REST :

- `T1595 - Active Scanning` : l'attaquant cartographie votre API publique (Nuclei, ffuf).
- `T1190 - Exploit Public-Facing Application` : il exploite une injection SQL (sous-technique `T1190.001`).
- `T1078 - Valid Accounts` : il récupère des credentials, se connecte légitimement.
- `T1552 - Unsecured Credentials` : il trouve un token AWS dans une variable d'environnement.
- `T1486 - Data Encrypted for Impact` : il exfiltre puis chiffre vos données pour rançon.

Cette *kill chain* permet de concevoir des détections à chaque étape (logs nginx anormal, anomalie sur `/api/v1/auth/login`, exfiltration massive sortante).

1.3.4 Frameworks moins centraux mais utiles à connaître

- **NIST 800-30** : guide officiel du National Institute of Standards and Technology pour la conduite d'évaluations de risques. Méthodologie très formelle, exigée dans le secteur public américain.
- **OCTAVE Allegro** : approche développée par l'université Carnegie Mellon, plus légère, orientée actifs informationnels (à quoi tient la valeur de l'organisation, qui voudrait s'en prendre à elle).
- **TRIKE** : framework axé sur l'*audit* : on part d'un modèle de droits attendus et on vérifie l'écart avec ce que le système autorise réellement.
- **VAST** (*Visual, Agile, and Simple Threat modeling*) : intégration explicite dans les workflows agiles, soutenu par l'outil ThreatModeler.

En résumé. Si vous ne devez en retenir que trois : STRIDE pour identifier les menaces sécurité, LINDDUN pour identifier les risques vie privée, ATT&CK pour relier vos défenses aux attaques réelles observées sur le terrain.

2 Le top 10 des risques OWASP API (2023)

2.1 API1 – BOLA (Broken Object Level Authorization)

Définition

BOLA : le serveur expose un identifiant (id, slug, UUID) sans vérifier que l'appelant a le droit d'accéder à cet objet précis. L'attaquant change l'id dans l'URL et obtient des données qui ne lui appartiennent pas.

Exemple

Vulnérable :

```
app.get("/notes/:id", auth, async (req, res) => {
  const note = await db.notes.findById(req.params.id);
  res.json(note);
});
// GET /notes/42 -- l'utilisateur 7 lit la note de l'utilisateur 3
```

Sûr :

```
app.get("/notes/:id", auth, async (req, res) => {
  const note = await db.notes.findOne({
    _id: req.params.id,
    ownerId: req.user.id // <-- vérification d'appartenance
  });
  if (!note) return res.status(404).end(); // <-- 404, pas 403
  res.json(note);
});
```

Pourquoi 404 et non 403 ? Renvoyer 403 confirme l'existence de la ressource et permet l'*énumération*. Renvoyer 404 dans tous les cas (n'existe pas / n'appartient pas) ne donne aucune information à l'attaquant.

2.2 API3 – BOPLA (Mass assignment et Excessive data exposure)

Définition

Mass assignment : l'API accepte d'écrire un champ qu'elle ne devrait pas (typiquement `role`, `is_admin`, `owner_id`).

Excessive data exposure : l'API renvoie tous les champs de l'objet, y compris des champs sensibles (hash de mot de passe, notes internes), en comptant à tort sur le client pour les filtrer.

Mitigation. DTO (*Data Transfer Object*) explicites en entrée *et* en sortie. `additionalProperties: false` dans le schéma JSON.

2.3 API7 – SSRF (Server-Side Request Forgery)

Définition

SSRF : l'API émet des requêtes pour le compte de l'attaquant. Scénario typique : un endpoint « importer depuis URL » qui reçoit une URL contrôlée par l'attaquant et la suit.

Cibles classiques (cloud) :

- AWS : `169.254.169.254/latest/meta-data/iam/security-credentials/` pour voler les credentials IAM.
- GCP : `metadata.google.internal/computeMetadata/v1/`.
- Services internes : Redis, Memcached, panneaux d'admin sur `localhost`.

Mitigation correcte :

- Liste blanche stricte des domaines/protocoles autorisés.
- Bloquer les IPs internes (RFC 1918, 127/8, 169.254/16).
- Désactiver les redirections automatiques.
- Sur AWS : forcer *IMDSv2*.

2.4 API4 – Unrestricted Resource Consumption

Définition

Unrestricted Resource Consumption : pas de rate limiting, pagination max absente, requêtes coûteuses non plafonnées, regex catastrophique (*ReDoS*), upload sans limite. Une rafale d'attaquant suffit à mettre l'API à genoux.

Mitigation. Rate limiting (token bucket par IP/clé), pagination obligatoire, timeouts agressifs, body size limit, scan des regex.

3 Attaques côté navigateur

3.1 XSS (Cross-Site Scripting)

Définition

XSS : du code JavaScript attaquant s'exécute dans le contexte d'un utilisateur authentifié et peut voler ses tokens, son cookie de session, faire des requêtes en son nom.

Trois variantes.

- *Stored XSS* : le code est stocké côté serveur (BDD) et servi à tous les visiteurs.
- *Reflected XSS* : le code arrive en paramètre et est renvoyé tel quel dans la page.
- *DOM-based XSS* : le code est injecté côté client via du JS qui manipule le DOM sans échappement.

Mitigation. Échappement contextuel en sortie (HTML, attribute, JS, URL), CSP stricte, cookies HttpOnly.

3.2 CSRF (Cross-Site Request Forgery)

Définition

CSRF : un site malveillant déclenche une requête vers votre API en utilisant les cookies de session de la victime authentifiée.

Mitigation.

- Cookies `SameSite=Lax` ou `Strict` (couvre 95 % des cas).
- Token CSRF (synchronisé ou *double submit*) pour les flows à risque.
- Vérifier l'en-tête `Origin` ou `Referer`.

3.3 CORS mal configuré

Définition

CORS (*Cross-Origin Resource Sharing*) : mécanisme du navigateur qui autorise des appels HTTP depuis une origine différente. Configuré côté serveur via les en-têtes `Access-Control-Allow-*`.

L'erreur la plus dangereuse : `Access-Control-Allow-Origin: *` avec `Allow-Credentials: true`. Cette combinaison est interdite par la spec mais des serveurs naïfs l'autorisent quand même.

Règles d'or : liste blanche d'origines, jamais de wildcard avec credentials, ne pas réfléchir l'`Origin` sans validation.

4 Mauvaises configurations classiques

À retenir

Les défauts de configuration sont la 8e catégorie OWASP API. Les plus fréquents :

- Headers de sécurité absents (HSTS, CSP, X-Content-Type-Options).
- CORS trop large.
- Mode debug actif en production (stack traces, panneaux d'admin).
- Verbose errors qui exposent SQL, framework, versions.
- Endpoints fantômes (`/api/v0`, `/api/internal`, `/admin/console`).
- Secrets en dur dans le code ou les variables d'environnement exposées.
- Dépendances obsolètes avec CVE connues.

5 CVSS : noter une vulnérabilité

Définition

CVSS (*Common Vulnerability Scoring System*, FIRST.org) : système de notation standardisé des vulnérabilités. Un score de 0 à 10 et une qualification (None, Low, Medium, High, Critical).

Vecteur composé de 8 métriques de base :

- AV (*Attack Vector*) : Network, Adjacent, Local, Physical.

- AC (*Attack Complexity*) : Low, High.
- PR (*Privileges Required*) : None, Low, High.
- UI (*User Interaction*) : None, Required.
- S (*Scope*) : Unchanged, Changed.
- C, I, A (*Confidentiality, Integrity, Availability impact*) : None, Low, High.

Exemple

Un BOLA classique : `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` → score 6,5 (Medium).
Si l'attaque permet aussi de modifier (`I:H`) → 8,1 (High).
Si elle permet d'atteindre d'autres tenants (`S:C`) → 9,3 (Critical).

6 Les outils d'audit

6.1 OWASP ZAP

Outil

OWASP ZAP (*Zed Attack Proxy*, https://www.zaproxy.org) est un scanner de vulnérabilités web open source édité par l'OWASP. Gratuit, multi-plateforme, scriptable.

À quoi ça sert ?

- Proxy interceptant entre le client et l'API, pour observer et modifier le trafic.
- Scan automatique passif (analyse du trafic) ou actif (envoi de payloads).
- Spider qui découvre la surface d'attaque.
- Import OpenAPI/Swagger pour cibler tous les endpoints.
- Mode automation pour intégration CI/CD (`zap-baseline.py`, image Docker).

Quand l'utiliser ? Pour la couverture automatisée en CI/CD et les premiers audits.

6.2 Burp Suite

Outil

Burp Suite (https://portswigger.net/burp) est l'outil de référence des *pen-testers* web. Édité par **PortSwigger**, société britannique fondée en 2008 par Dafydd Stuttard (l'un des co-auteurs du livre *The Web Application Hacker's Handbook*). Existe en version Community gratuite et Pro payante.

À quoi ça sert ?

- *Proxy* : intercepter, modifier, rejouer chaque requête HTTP.
- *Repeater* : modifier puis renvoyer une requête en isolation.
- *Intruder* : fuzzer un paramètre avec une liste de payloads.
- *Decoder, Comparer, Sequencer* : utilitaires de manipulation.
- *Extender + BAppStore* : extensions (voir ci-dessous).

Quand l'utiliser ? Pour l'investigation manuelle ciblée et le pen-test approfondi. Complémentaire de ZAP.

Extensions Burp populaires (BAppStore) :

- *JWT Editor* : décoder, modifier et re-signer un JWT directement dans Burp ; tester en

un clic les attaques classiques (`alg=none`, confusion HS/RS, faiblesse de signature).

- *Authorize* : rejouer automatiquement chaque requête avec une seconde session (un autre utilisateur) et signaler celles qui retournent les mêmes données — détection rapide de BOLA et BFLA.
- *Logger++* : remplace le *Logger* natif par un journal avancé filtrable, exportable, indispensable pour les audits longs.

6.3 Burp Active Scan — au sein de Burp Suite

Outil

Burp Active Scan : module de *Burp Suite Professional* qui exécute automatiquement une batterie de tests d'attaques (injections SQL, XSS, SSRF, headers, désérialisation, etc.) sur les requêtes capturées par le proxy. Inclus uniquement dans la version Pro.

Comment l'utiliser ? On navigue dans l'application via le proxy Burp, puis clic droit sur une requête → *Do active scan*. Burp rejoue la requête en injectant des centaines de payloads et signale les anomalies (réponse différente, temps anormal, code d'erreur, message révélateur).

Exemple : sur `POST /search` avec body `{"q":"test"}`, Burp testera `q=' OR 1=1 -`, `q=<script>alert(1)</script>`, `q=${jndi:ldap://...}`, etc. Trouve la SQLi en moins de 30 secondes.

6.4 Outils de scan complémentaires

Outil

Nuclei (`https://nuclei.projectdiscovery.io`) : scanner de vulnérabilités open source édité par ProjectDiscovery, écrit en Go. Sa particularité est d'utiliser des *templates YAML* (plus de 9 000 disponibles publiquement) qui décrivent chacun un test précis (CVE, mauvaise configuration, exposition).

Pourquoi c'est utile ? Très rapide (parallélisation native), nettement plus précis qu'un scanner générique, et la communauté met à jour les templates dès qu'une nouvelle CVE est publiée.

Exemple :

```
# Tester un site contre tous les templates "exposures"
nuclei -u https://api.example.com -t exposures/

# Tester une CVE specifique sur un parc
nuclei -l urls.txt -t cves/2024/CVE-2024-12345.yaml
```

Outil

Nikto (`https://github.com/sullo/nikto`) : scanner web open source historique (depuis 2001) écrit en Perl par Chris Sullo. Détecte des centaines de mauvaises configurations courantes (fichiers oubliés, headers manquants, versions vulnérables).

Quand l'utiliser ? Premier scan « éclair » d'un serveur web pour identifier les évidences.

Aujourd'hui souvent remplacé par Nuclei mais reste répandu.

Exemple : `nikto -h https://api.example.com -o nikto.html -Format html`.

Outil

ffuf (*Fuzz Faster U Fool*, https://github.com/ffuf/ffuf) : outil de fuzzing HTTP très rapide écrit en Go. Sert principalement à **énumérer des chemins ou paramètres cachés** en rejouant une requête avec une liste de mots à la place d'un `FUZZ`.

Exemple — découvrir des endpoints cachés :

```
ffuf -u https://api.example.com/FUZZ \
     -w /usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt \
     -mc 200,401,403 # afficher seulement ces codes
```

Résultat type : `/admin (403)`, `/internal (401)`, `/swagger.json (200)` — toutes des découvertes intéressantes pour la suite de l'audit.

Outil

dirsearch (https://github.com/maurosoria/dirsearch) : scanner de chemins web en Python, plus convivial que ffuf pour les débutants. Wordlists par défaut intelligentes, sortie colorée et structurée.

Exemple : `dirsearch -u https://api.example.com -e php,html,json -t 50`.

Outil

gobuster (https://github.com/OJ/gobuster) : équivalent Go de dirsearch, ultra-rapide. Sait aussi énumérer sous-domaines DNS et buckets S3.

Exemple :

```
gobuster dir -u https://api.example.com \
             -w /usr/share/seclists/Discovery/Web-Content/common.txt
gobuster dns -d example.com \
             -w /usr/share/seclists/Discovery/DNS/subdomains-top1million.txt
```

Outil

httpx (ProjectDiscovery, https://github.com/projectdiscovery/httpx) : sondeur HTTP massivement parallèle. Prend une grande liste d'URLs/IPs en entrée et sort, pour chacune, le code de statut, le titre, la techno détectée, le hash favicon, etc.

Exemple — inventaire d'une surface d'attaque :

```
cat sous_domaines.txt | httpx -title -status-code -tech-detect
# https://api.example.com [200] [Login] [nginx, Express]
# https://admin.example.com [403] [] [nginx]
# https://staging-api.example.com [200] [Welcome] [Apache, PHP/7.4]
```

Outil

sqlmap (https://sqlmap.org) : outil open source de référence pour automatiser *l'exploitation* (et non juste la détection) des injections SQL. Une fois la faille trouvée,

sqlmap peut extraire la base, dumper des tables, voire ouvrir un shell sur le serveur.

Exemple :

```
sqlmap -u "https://api.example.com/products?id=1" \
       --dbs --batch
# [+] available databases [3]: information_schema, app, app_test
```

Note : à n'utiliser que sur un système qu'on possède ou avec autorisation écrite. Très bruyant dans les logs.

Outil

Trivy (https://aquasecurity.github.io/trivy/, Aqua Security) : scanner de vulnérabilités tout-en-un, open source, écrit en Go. Détecte les CVE dans les *images container* (Docker, OCI), les *dépendances* (npm, pip, Maven, etc.), les *configurations* (Kubernetes, Terraform) et les *secrets* (clés AWS, tokens).

Exemple :

```
trivy image my-api:1.4.2 # scanner une image Docker
trivy fs ./ # scanner un dépôt code
trivy config ./k8s # scanner des manifests K8s
```

Outil

Grype (https://github.com/anchore/grype, Anchore) : alternative à Trivy, focalisée sur le scan d'images et de dépendances. Souvent utilisée en duo avec **Syft** (génération de SBOM).

Exemple :

```
grype my-api:1.4.2 # liste des CVE par sévérité
syft my-api:1.4.2 -o spdx-json # SBOM standardisé
```

Outil

Semgrep (https://semgrep.dev, Semgrep Inc.) : analyseur statique **(SAST)** multi-langage, open source, où l'on écrit ses règles dans une syntaxe proche du code visé. Détecte les patterns dangereux (concaténation SQL, secrets hardcodés, désérialisation non sûre).

Exemple — détecter une concaténation SQL en Node :

```
# règle Semgrep
rules:
  - id: sql-concat
    pattern: db.query("..." + $X + "...")
    message: "Concaténation SQL avec variable - utiliser des paramètres"
    severity: ERROR
    languages: [javascript]
```

Lancement : `semgrep -config p/security-audit ./.`

Outil

CodeQL (https://codeql.github.com, GitHub) : moteur d'analyse statique très puissant qui transforme le code en base de données interrogeable par un langage dédié (similaire à SQL). Intégré gratuitement à GitHub Advanced Security pour les dépôts open source.

Exemple — requête CodeQL pour SSRF en JavaScript :

```
import javascript
import semmle.javascript.security.dataflow.RequestForgery::Configuration
from Configuration cfg, DataFlow::PathNode source, DataFlow::PathNode sink
where cfg.hasFlowPath(source, sink)
select sink.getNode(), "Possible SSRF: donnée utilisateur atteint un fetch HTTP."
```

6.5 Plateformes vulnérables pour s'entraîner légalement

Outil

VAmpAPI (https://github.com/erev0s/VAmPI, par **Erev0s**, alias de Konstantinos Papapanagiotou, consultant sécurité grec) : application API REST volontairement vulnérable, écrite en Python/Flask. Conçue spécifiquement pour s'entraîner aux 10 risques OWASP API. Image Docker prête à l'emploi : `erev0s/vampi:latest`.

Outil

crAPI (*Completely Ridiculous API*, https://github.com/OWASP/crAPI, OWASP) : application API moderne et complète, simulant une plateforme de gestion de véhicules avec utilisateurs, mécaniciens, communauté. Couvre les 10 OWASP API dans un scénario plus réaliste que VAmpAPI.

Outil

DVWA (*Damn Vulnerable Web Application*, https://dvwa.co.uk) : application PHP/MySQL volontairement vulnérable créée en 2010. Niveaux de difficulté progressifs (*low*, *medium*, *high*, *impossible*). Idéale pour comprendre concrètement chaque type d'injection et de XSS.

Exemple d'usage : on choisit le module *SQL Injection*, niveau *low*. Le formulaire interroge `SELECT * FROM users WHERE id='$id'`. On entre `1' OR '1'='1` et on récupère tous les utilisateurs. Au niveau *impossible*, le code utilise des requêtes paramétrées : la même attaque échoue, on apprend la bonne pratique.

Outil

WebGoat (https://owasp.org/www-project-webgoat/, OWASP) : application Java pédagogique avec leçons interactives guidées (XSS, CSRF, JWT, désérialisation, accès…). Chaque leçon explique la vulnérabilité, propose un exercice et un corrigé.

Outil

OWASP Juice Shop (https://owasp.org/www-project-juice-shop/) : SPA Node.js / Angular volontairement vulnérable, gamifiée (challenges à débloquer, scoreboard). Probablement la plus utilisée dans le monde pour les ateliers d'initiation à la sécurité offensive.

Outil

PortSwigger Web Security Academy (`https://portswigger.net/web-security`) : plateforme de formation gratuite éditée par PortSwigger (les créateurs de Burp Suite). Plus de 250 labs interactifs, classés par catégorie (SQLi, XSS, JWT, OAuth, race conditions…), avec corrigés écrits et vidéos.

À savoir : la qualité pédagogique est excellente. C'est aujourd'hui la référence pour apprendre la sécurité web sérieusement.

Outil

HackTheBox (`https://www.hackthebox.com`) et **TryHackMe** (`https://tryhackme.com`) : plateformes commerciales (avec offres gratuites) qui proposent des *machines virtuelles vulnérables* en mode CTF (*Capture The Flag*). Le but : pénétrer la machine, escalader les privilèges, récupérer un drapeau caché.

Différence : TryHackMe est plus pédagogique (parcours guidés), HackTheBox plus orienté pen-test réaliste (peu d'indices, beaucoup de cherche et débrouille).

7 Démarche d'audit en équipe

1. **Cadrage** : périmètre, environnement (préprod jamais prod), contacts, *rules of engagement* (« règles d'engagement » : ce qui est autorisé, fenêtres horaires, données sensibles à éviter).
2. **Reconnaissance** : récupération de la spec *OpenAPI*, lecture du fichier `robots.txt`, énumération de sous-domaines, scans passifs (sans envoi de payloads agressifs).
3. **Mapping** : cartographier toutes les routes, paramètres et schémas d'authentification de l'application.
4. **Scan automatisé** : couvrir la surface avec ZAP (mode *baseline* ou *full*), *Burp Active Scan* (cf. ci-dessous) et Nuclei.
5. **Tests manuels** : passer en revue les 10 OWASP API, focus particulier sur la logique métier (qu'aucun scanner ne sait tester).
6. **Reporting** : un *finding* par vulnérabilité avec titre, sévérité CVSS, vecteur, étapes de reproduction, correctif proposé.
7. **Restitution** : présentation orale + livrable écrit, classé par criticité.
8. **Rétest** après corrections, pour valider que les findings ont bien été résolus sans en introduire de nouveaux.

8 Aspects légaux à connaître

À retenir

Avertissement. On ne teste *jamais* une API qu'on ne possède pas, sans autorisation écrite du propriétaire.

Définition

Loi Godfrain : loi française du 5 janvier 1988, du nom du député Jacques Godfrain qui l'a portée. Elle a introduit dans le Code pénal les articles 323-1 à 323-7 qui sanctionnent les

atteintes aux *Systèmes de Traitement Automatisé de Données (STAD)*. Peines actuelles : jusqu'à 3 ans de prison et 100 000 EUR d'amende pour l'accès ou le maintien frauduleux ; 5 ans et 150 000 EUR si modification ou suppression de données ; 7 ans et 300 000 EUR pour des STAD de l'État.

8.1 Plateformes de bug bounty

Définition

Bug bounty : programme par lequel une organisation invite publiquement (ou en privé) des chercheurs en sécurité à signaler des vulnérabilités contre récompense (financière ou en nature). Les règles définissent strictement le périmètre autorisé et les méthodes acceptées : tester en dehors du *scope* reste illégal.

Outil

HackerOne (`https://www.hackerone.com`) : plateforme américaine fondée en 2012, leader mondial. Anime les programmes de Google, Twitter/X, Microsoft, Goldman Sachs, US Department of Defense. Récompenses pouvant aller jusqu'à 100 000 $ par faille critique.

Exemple d'usage : créer un compte gratuit, choisir un programme *public* (ex. Shopify), lire le scope (ce qui est autorisé), tester, soumettre un rapport. Si validé, on est rémunéré et on apparaît au classement public.

Outil

Bugcrowd (`https://www.bugcrowd.com`) : plateforme américaine fondée en 2012. Concurrent direct de HackerOne, légèrement plus orienté entreprises moyennes. Anime notamment les programmes de Tesla, Mastercard, Atlassian.

Outil

YesWeHack (`https://www.yeswehack.com`) : plateforme française fondée en 2015 (Rouen), leader européen du bug bounty. Anime des programmes pour OVH, Doctolib, Orange, BlaBlaCar, ainsi que pour le ministère des Armées français. Souvent privilégiée par les organisations européennes pour les questions de souveraineté des données.

Pour s'entraîner sans même chercher de programme bug bounty, utiliser les applications volontairement vulnérables (VAmpAPI, crAPI, Juice Shop) ou les labs PortSwigger Web Security Academy — toutes décrites dans la section *Plateformes vulnérables*.

9 Synthèse

À retenir

- OWASP API Top 10 : 9 risques sur 10 portent sur l'authentification, l'autorisation ou la validation.
- BOLA = n°1. Toujours filtrer par *owner*. Renvoyer 404 et non 403.
- DTO explicites en entrée et en sortie pour empêcher mass assignment et excessive data exposure.

- SSRF : whitelist d'URLs, blocage des IPs internes, IMDSv2 sur AWS.
- ZAP en CI pour la couverture automatisée, Burp en pen-test pour l'investigation manuelle.
- Modéliser les menaces (STRIDE) en amont, pas en réaction.

10 Lexique du module

BFLA Broken Function Level Authorization. Endpoint admin accessible par un user normal.

BOLA Broken Object Level Authorization. Accès à un objet qui ne nous appartient pas.

BOPLA Broken Object Property Level Auth. Mass assignment ou excessive data exposure.

Bug bounty Programme public où une organisation rémunère ceux qui rapportent des failles.

CSRF Cross-Site Request Forgery.

CSP Content Security Policy. Header navigateur de défense XSS.

CVE Common Vulnerabilities and Exposures. Identifiant unique d'une vulnérabilité publique.

CVSS Common Vulnerability Scoring System.

DAST Dynamic Application Security Testing (scan dynamique).

Énumération Découverte d'identifiants ou ressources existantes par essais successifs.

Excessive data exposure L'API renvoie trop de champs.

Finding Vulnérabilité identifiée lors d'un audit.

IDOR Insecure Direct Object Reference. Synonyme de BOLA.

Injection Famille d'attaques où une donnée est interprétée comme du code.

LINDDUN Framework de threat modeling pour la vie privée.

Mass assignment Écriture de champs interdits.

MITRE ATT&CK Taxonomie d'attaques réelles.

OWASP Open Web Application Security Project.

Pen-test Test d'intrusion mandaté.

Reconnaissance Phase d'audit consistant à cartographier la cible.

ReDoS Regular Expression Denial of Service.

SAST Static Application Security Testing (analyse statique).

SBOM Software Bill of Materials. Liste des composants d'un logiciel.

Spider Robot qui explore automatiquement les liens d'un site.

SSRF Server-Side Request Forgery.

STRIDE Modèle Microsoft de threat modeling.

Threat modeling Démarche d'identification des menaces avant le code.

XSS Cross-Site Scripting.

11 Pour aller plus loin

- OWASP API Security Top 10 (2023).
- OWASP Web Security Testing Guide.
- Adam Shostack, *Threat Modeling : Designing for Security*.
- Daniel Stuttard & Marcus Pinto, *The Web Application Hacker's Handbook*.
- PortSwigger Web Security Academy (`https://portswigger.net/web-security`).
- NIST SP 800-204 (Microservices security).